

Final Project

ECEN 2233 – Fundamentals of Digital Logic Design

Spring 2026

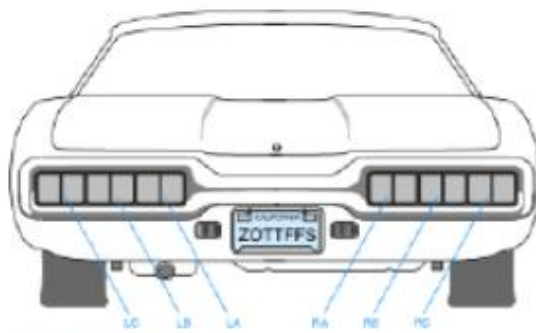
Project Title: FPGA-Based Automotive Taillight Controller Using FSM

Project Overview:

In this project, you will design and implement a **Finite State Machine (FSM)** to control the taillights of a Ford Thunderbird. The goal is to simulate realistic automotive lighting behavior using digital logic on an FPGA platform.

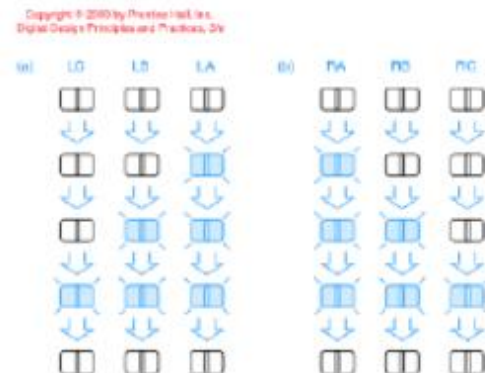
This project demonstrates how **sequential logic (memory elements)** and **combinational logic** work together to produce controlled and time-dependent behavior.

The FSM stores the current state of the system and transitions between states based on input signals such as **reset, left, right, and brake**.



Copyright © 2009 by Prentice Hall, Inc.
Digital Design: Principles and Practices, 2/e

Thunderbird Tail Lights [1]



Flashing Sequence (shaded lights are illuminated) [1]

System Description

Each side of the vehicle contains **three lights**:

- Left side: LA, LB, LC
- Right side: RA, RB, RC

The FSM generates different lighting patterns depending on the driver's input.

Inputs and Outputs

Inputs:

- **reset**: Resets the system (all lights OFF)
- **left**: Activates left turn signal
- **right**: Activates right turn signal
- **brake**: Activates brake lights

Outputs:

- **lights[5:0]**: Controls the six taillights
- **7-segment display**: Displays the current mode

System Behavior

Reset Mode: When the system is reset:

- All lights are OFF
- FSM returns to the **IDLE state**

Left Turn Mode

When the **left** input is activated:

The left-side lights turn on sequentially:

1. LA
 2. LA + LB
 3. LA + LB + LC
- The sequence continues even if the button is released mid-pattern
 - If the **left** input remains active, the sequence **repeats continuously**

Right Turn Mode

When the **right** input is activated:

The right-side lights behave similarly:

1. RA
 2. RA + RB
 3. RA + RB + RC
- The sequence repeats as long as **right** remains active

Hazard Mode (Emergency Lights)

When both **left** and **right** are active simultaneously:

The system enters **hazard mode**, where both sides flash together in sequence:

1. LA + RA
 2. LA + LB + RA + RB
 3. LA + LB + LC + RA + RB + RC
- The pattern repeats continuously
 - This mimics realistic hazard light behavior

Brake Mode

When the **brake** input is activated:

- All lights turn ON simultaneously
- Lights remain ON as long as brake is active

Priority Handling (Very Important)

The system implements **priority-based control** to ensure realistic behavior:

Brake > Hazard > Turn Signals (Left/Right)

This means:

- If **brake** is active → system ignores all other inputs
- If **left** and **right** are both active → hazard mode is selected
- If only one is active → corresponding turn signal is used

Example:

- If **brake** and **left** are both pressed → system stays in **brake mode**

FSM Design

The system is implemented using a **Moore-type FSM**, where:

- Outputs depend only on the current state
- Each state represents a step in the lighting sequence

The FSM transitions between states at each clock cycle.

Clock Control

Since the FPGA clock is very fast, a **clock divider** is used to slow down the signal so that the lighting sequence is visible to the human eye.

7-Segment Display

A 7-segment display is used to show the current system mode:

Mode	Display
Left	L
Right	r
Hazard	H
Brake	b
Idle	OFF

This feature helps with:

- Debugging
- Visualization
- User interaction

Simulation and Verification

Before hardware implementation:

- The system is simulated using **ModelSim**
- A **testbench** is created to test all input combinations
- Waveforms are analyzed to verify:
 - Correct state transitions
 - Proper timing
 - Correct output sequences
 - Priority behavior

FPGA Implementation

The design is implemented on an FPGA with:

- Push buttons as inputs
- LEDs for taillight outputs
- 7-segment display for mode visualization

All signal names are matched with the **constraints file** to ensure correct hardware mapping.

Total Points: 300

- SystemVerilog Design & Simulation (ModelSim): **100 Points**
- FPGA Implementation (Vivado + Xilinx Board): **120 Points**
- Lab Report: **80 Points**

Submission

Submit the following files:

- top_thunderbird.sv
- thunderbird_fsm.sv
- seven_seg_decoder.sv
- clock_divider.sv
- tb_thunderbird.sv
- Lab report (PDF)

Submit all files as a **single .zip file** on Canvas

Grading Notes

- Code must be **clean, readable, and modular**
- All modules must be included
- Design must be **fully functional on FPGA**
- Demonstration required for full credit